加速开发，释放生产力的必备利器

作者 李东锋、肖婷、周奇龙、黄鑫

时间 2026/03

目录

Part 1 算子开发旅程

Part 2 Cube算子模板库 CATLASS

Part 3 Vector算子模板库 ATVOSS

Part 4 <<<>>>直调

Part 5 仿真工具

算子开发旅程

	前置操作	算子设计与实现	算子编译运行	算子调试调优
KEP Key Experience Path	资料获取 目标内容定位 环境准备	参考示例代码 算子设计（需求分析、方案设计） 算子代码实现	编译配置 算子编译	功能/精度调测 运行调优工具，获取性能数据 分析性能瓶颈
用户痛点问题	环境部署困难	参考样例零散无体系 Tiling实现复杂 底层指令上手难度大	编译配置复杂	上板环境难获得 调试调优效率低
必备利器		算子模板库	<<<>>>直调	调测工具

https://gitcode.com/cann

CANN

目录

Part 1 算子开发旅程

Part 2 Cube算子模板库 CATLASS

Part 3 Vector算子模板库 ATVOSS

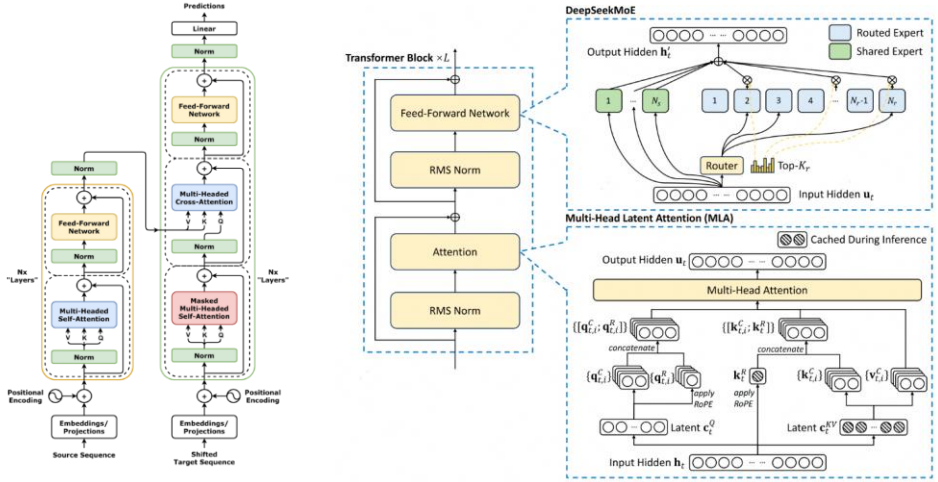
Part 4 <<<>>>直调

Part 5 仿真工具

问题背景：GEMM类算子开发难度较高，开发周期长

问题背景

Transformer架构特点：当前大模型基于Transformer架构设计，**GEMM相关计算(Matmul/FA/Conv及其融合)**占据其中主要运算部分，优化其性能对提升整体系统效率至关重要



GEMM类算子开发存在难点

- 形状变种多
- 功能变种多
- 底层指令学习难度大

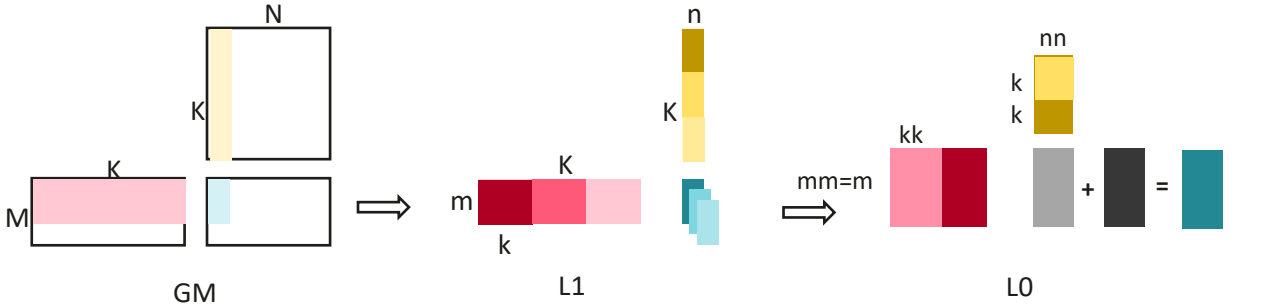
核心诉求

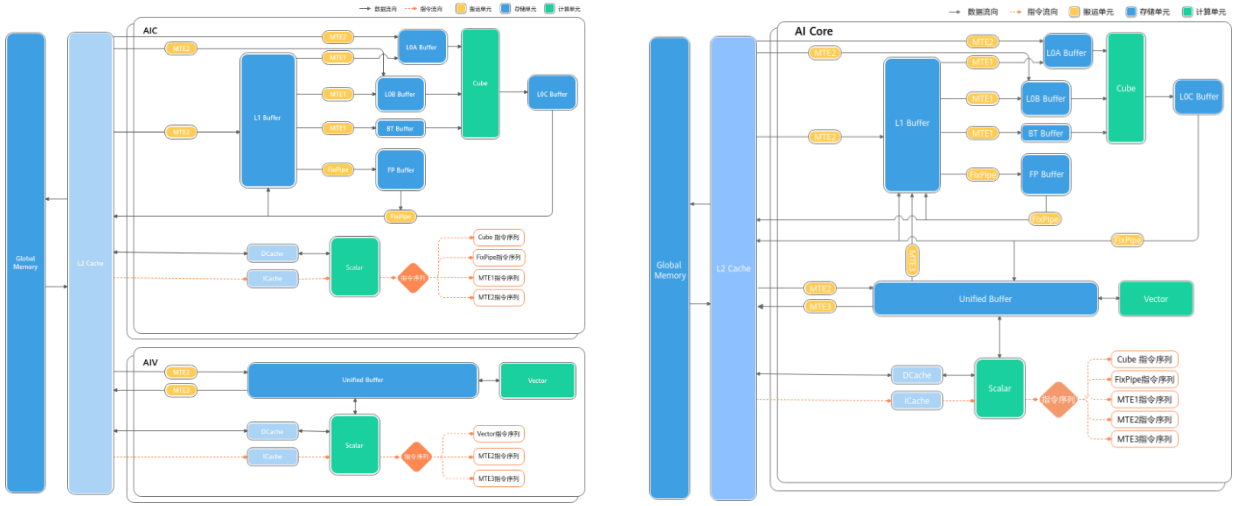
模块拆解可复用

提取共性，屏蔽差异

高效开发

极致性能定制算子





https://www.hiascend.com/document/detail/zh/CANNCommunityEdition/83RC1/opdevg/Ascendcopdevg/atlas_ascendc_10_0011.html

解决方案：推出CATLASS算子模板库，提升GEMM开发效率

CATLASS： **CA**NN **T**emplates for **L**inear **A**lgebra **S**ubroutines

按照计算粒度，提供各层级源码，支持灵活组合

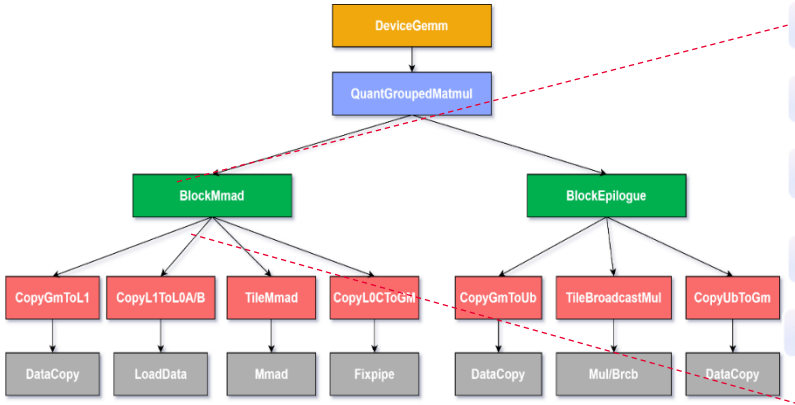
计算粒度：大 → 小

调用接口	Device	DeviceGemm	DeviceGemv	…		
多核计算	Kernel	MatmulKernel	FAKernel	…		
单核计算	Block	前处理	矩阵乘累加	后处理		
单个步骤	Tile	Gm到L1数据搬运	L1到L0数据搬运	L0C到UB数据搬运	L1到UB数据搬运	…
单条指令	Basic	DataCopy	Load2D	Load3D	VAdd	VSub

前 → 执行顺序 → 后

https://gitcode.com/cann

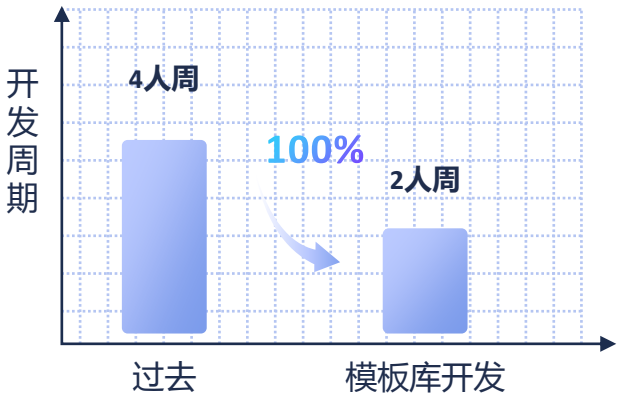
模块化设计，分层复用



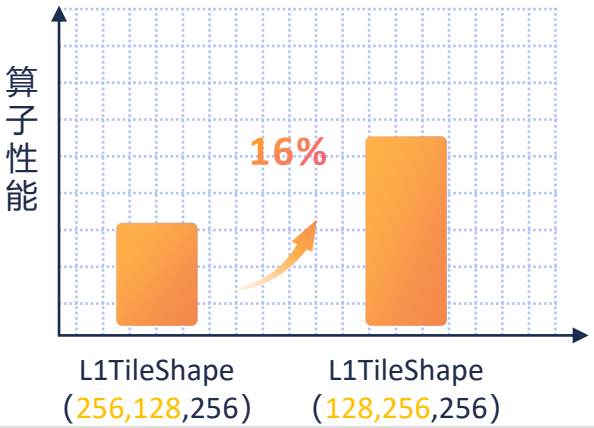
- L1/L0上的多buffer
- k循环读取shuffle
- 大K场景下k维度分核
- 小shape下L1常驻
- 混合内外积消解MTE2瓶颈
- … …

支撑开发效率与优化效率提升

Matmul算子开发效率提升1倍



自定义修改参数提升算子性能



CATLASS模板库的实现：算子模板如何拼装

基础使用：组件快速拼装

Matmul+Add拼接组装

```
using ArchTag = Arch::Ascend950;
// Block level, define BlockMmad
constexpr bool enableUnitFlag = true;
using L1TileShape = GemmShape<128, 256, 256>;
using L0TileShape = GemmShape<128, 256, 64>;
using AType = Gemm::GemmType<half, LayoutA>;
using BType = Gemm::GemmType<half, LayoutB>;
using CType = Gemm::GemmType<half, LayoutC>;
```

①指定平台、切分大小、数据类型

```
using MmadDispatchPolicy =
  Gemm::MmadPingpong<enableUnitFlag>;
using BlockMmad = Gemm::Block::BlockMmad<MmadDispatchPolicy, L1TileShape, L0TileShape, AType,
BType, CType>;
// Block level, define BlockEpilogue
using EpilogueDispatchPolicy = Epilogue::EpilogueElemWiseOneSource;
using XType = CType;
using DType = CType;
using ComputeType = CType;
constexpr uint32_t computeLength = 16384;
using TileElemWiseEpilogue = Epilogue::Tile::TileElemWiseAdd<ArchTag, ComputeType,
computeLength>;
using EpilogueTileCopy = Epilogue::Tile::TileCopy<ArchTag, CType, XType, DType>;
using BlockEpilogue = Epilogue::Block::BlockEpilogue<EpilogueDispatchPolicy, CType, XType, DType,
 TileElemWiseEpilogue, EpilogueTileCopy>;
std::vector<fp16_t> hostD(lenD);
// Define BlockScheduler
// Swizzle offset is 3 and direction is 0.
using BlockScheduler = typename Gemm::Block::GemmIdentityBlockSwizzle<3, 0>;
```

②配置矩阵乘累加与后融合策略

```
// Kernel level
 using MatmulKernel = Gemm::Kernel::MatmulEpilogue<BlockMmad, BlockEpilogue, BlockScheduler>;
// Prepare params
typename MatmulKernel::Arguments arguments{
 options.problemShape, sizeof(half), deviceA, deviceB, deviceD};
using MatmulAdapter = Gemm::Device::DeviceGemm<MatmulKernel>;
MatmulAdapter matmul_op;
```

③组装完成算子配置

进阶使用：实现按需定制

```
// Get the number of cube cores of the curren
auto aicCoreNum = platform_ascendc::Platfor
using ArchTag = Arch::Ascend950;
using DispatchPolicy = Gemm::MmadAtlasA2P
using L1TileShape = GemmShape<128, 256, 25

using L0TileShape = GemmShape<128, 256, 64

using AType = Gemm::GemmType<half, Layou
using BType = Gemm::GemmType<half, Layou
using CType = Gemm::GemmType<half, Layou

using BlockMmad = Gemm::Block::BlockMmad
CType>;
using BlockEpilogue = void;

// Swizzle offset is 3 and direction is 0.
using BlockScheduler = typename Gemm::Block::GemmIdentityBlockSwizzle<3, 0>;

// kernel level
using MatmulKernel = Gemm::Kernel::BasicMatmul<BlockMmad, BlockEpilogue, BlockScheduler>;

using MatmulAdapter = Gemm::Device::DeviceGemm<Matm
MatmulKernel::Arguments arguments{options.problemShape
MatmulAdapter matmul_op;
matmul_op.CanImplement(arguments);
size_t sizeWorkspace = matmul_op.GetWorkspaceSize(argun
uint8_t *deviceWorkspace = nullptr;
```

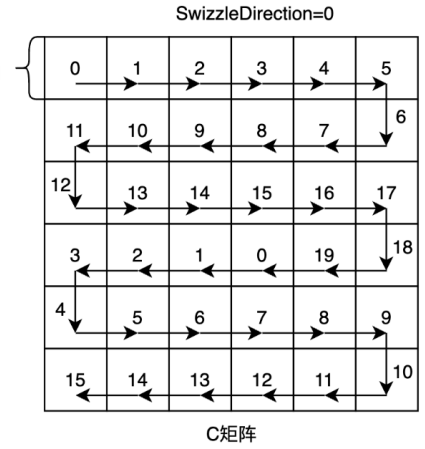
```
/// Block swizzling function for Gemms
template <uint32_t SwizzleOffset = 1, uint32_t SwizzleDirection = 0>
struct GemmIdentityBlockSwizzle {
    /// Data members

    GemmCoord problemShape;
    MatrixCoord tileMN;
    MatrixCoord loopsMN;

    /// Methods

    CATLASS_DEVICE
    GemmIdentityBlockSwizzle() {}

    CATLASS_DEVICE
    GemmIdentityBlockSwizzle(GemmCoord const &problemShape_, MatrixCoord const &tileMN_)
        : problemShape(problemShape_), tileMN(tileMN_)
    {
        loopsMN = CeilDiv(MatrixCoord(problemShape.GetCoordMN()), tileMN);
    }

    CATLASS_DEVICE
    GemmIdentityBlockSwizzle(GemmCoord const &problemShape_, MatrixCoord const &tileMN_,
        MatrixCoord const &loopsMN_)
        : problemShape(problemShape_), tileMN(tileMN_), loopsMN(loopsMN_) {}

    CATLASS_DEVICE
    void Update(GemmCoord const &problemShape_, MatrixCoord const &tileMN_)
    {
        problemShape = problemShape_;
        tileMN = tileMN_;
```

Swizzle模板类的定义

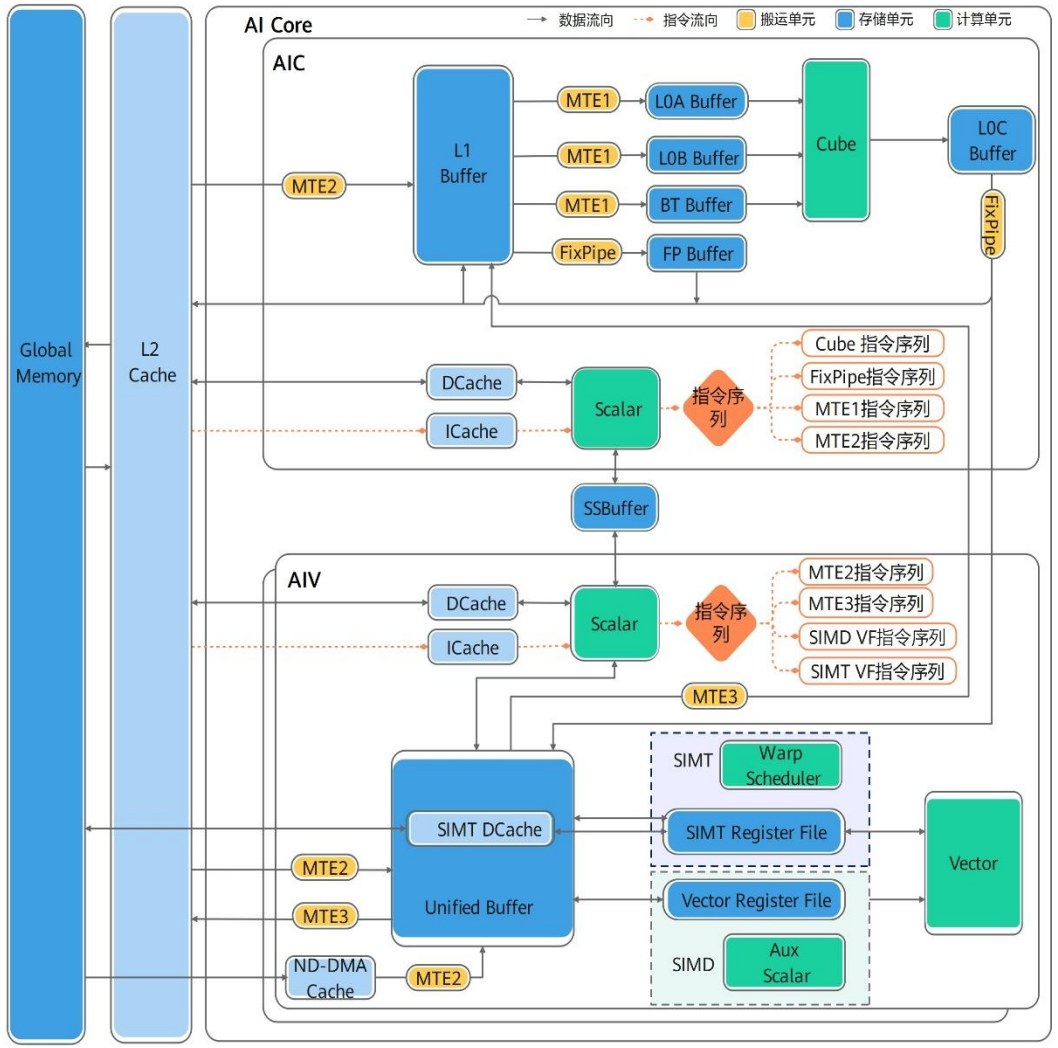
用户可以调用的接口,可修改的自定义内部实现

①按需编写遍历次序逻辑

②对接预留接口完成组装



Ascend 950架构概述



AIV：

- 可支持两种工作模式：SIMT和SIMD，以VF(Vector Fuction） 为粒度进行切换；同一时刻不能同时支持SIMD和SIMT；
- **SIMT**
  - 支持直接读写GM或UB (UB当做Shared Memory)
  - 一个SIMT Vector Function 运行一个 thread block，同一时刻一个AIV只能执行一个SIMT Vector Function
  - 每个AIV有4个Warp Scheduler, scheduler id = warp id % 4
  - 每个线程可使用的寄存器受线程个数限制，具体如右边所示
  - 最大支持**128KB** Data Cache， Data Cache直接复用UB作为Cacheline
- **SIMD**
  - 从A2/A3Memorybased架构切换到Regbased架构，增加SIMD Register File存储层次，可以直接对芯片的Vector寄存器Register进行操作，实现更大的灵活性和更好的性能。数据从Unified Buffer搬运到Register进行计算，产生的中间结果可以不用传回Unified Buffer，直接在寄存器计算；

AIC：

- 新增MX FP4/MX FP8类型；
- 增强单核算力，增大核数；
- 不再支持s4类型；不支持4:2稀疏矩阵；

数据通路：

- 增加L0C Buffer -> Unified Buffer、 Unified Buffer <-> L1 Buffer的数据通路；
- 删除Global Memory -> L0A Buffer、 Global Memory -> L0B Buffer的数据通路。删除L1 Buffer-> Global Memory的数据通路；
- MTE2 数据搬运能力增强：新增NDDMA搬运、新增Compact模式、增强搬运维度；
- SSBuffer，用于AIC和AIV的核间通信；

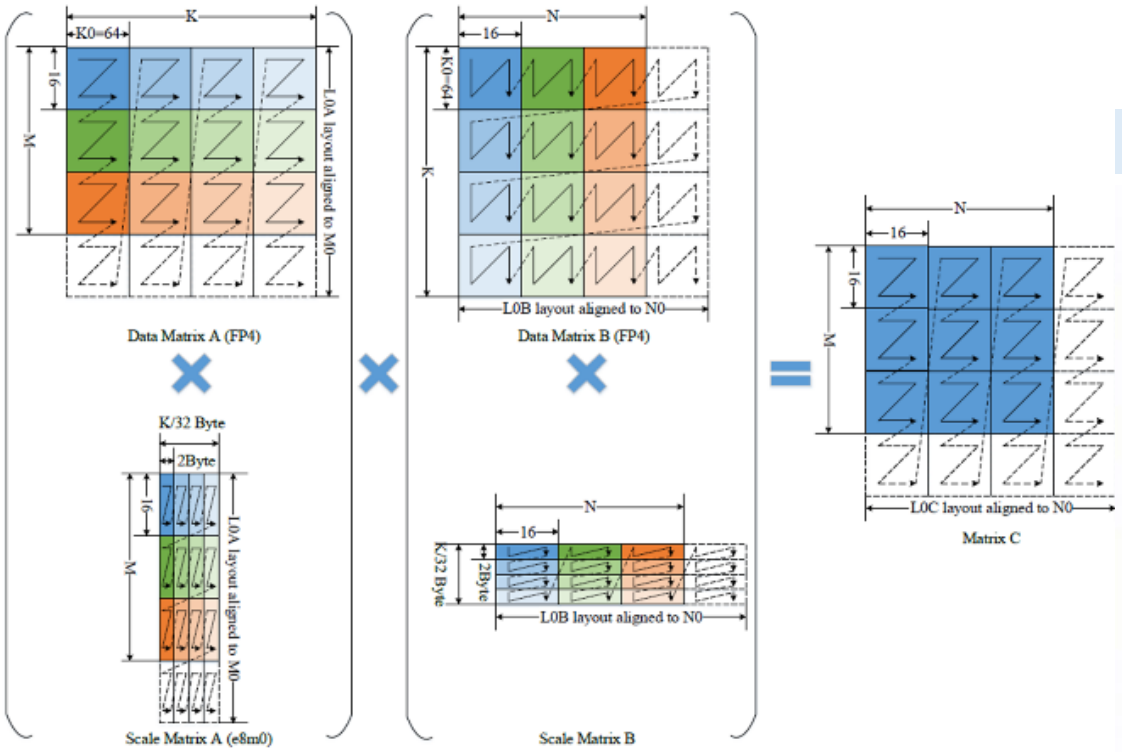
CANN

Ascend 950新特性支持: 新增低比特数据类型

➢ **以MXFP4为例**

A：[M, K], mxfp4(e2m1)

B：[K, N], mxfp4(e2m1)



ScaleA：[M, K/32], fp8(e8m0)

ScaleB：[K/32, N], fp8(e8m0)

MXFP4样例计算过程示意图

针对Ascend950芯片，支持包含缩放功能的矩阵乘，公式如下：

C = (ScaleA ⊗ A) ∗ (ScaleB ⊗ B) + C

ScaleA和ScaleB通过LoadData2DMX接口载入。

•ScaleA的分形格式为小Z大Z，shape为（16，2），数据类型为fp8_e8m0_t

•ScaleB的分形格式为小N大N，shape为（2，16），数据类型为fp8_e8m0_t

Tile层 代码片段：

```
AscendC::LoadData2DParamsV2 loadDataParams;
loadDataParams.mStartPosition = CeilDiv<C0_NUM_PER_FRACTAL>(tla::get<0>(srcCoord));
loadDataParams.kStartPosition = CeilDiv<ELE_NUM_PER_C0>(tla::get<1>(srcCoord));
loadDataParams.mStep = dstOuterShapeRow;
loadDataParams.kStep = dstOuterShapeCol;
loadDataParams.srcStride = CeilDiv<ELE_NUM_PER_FRACTAL>(srcOuterStrideCol);
loadDataParams.dstStride = CeilDiv<ELE_NUM_PER_FRACTAL>(dstOuterStrideCol);
loadDataParams.ifTranspose = false;

auto scaleCoord = scaleTensor.coord();
AscendC::LoadData2DMxParams loadDataMxParams;
loadDataMxParams.xStartPosition = CeilDiv<C0_NUM_PER_FRACTAL>(tla::get<0>(scaleCoord));
loadDataMxParams.yStartPosition = CeilDiv<MX_SCALE_COPY_GROUP_NUM>(tla::get<1>(scaleCoord));
loadDataMxParams.xStep = tla::get<0, 1>(scaleTensor.shape());
loadDataMxParams.yStep = tla::get<1, 1>(scaleTensor.shape());
loadDataMxParams.srcStride = CeilDiv<BYTE_PER_C0>(tla::get<0, 1>(scaleTensor.stride()));
loadDataMxParams.dstStride = tla::get<1, 1>(scaleTensor.shape());

auto dstOffset = dstTensor.layout()(dstTensor.coord());
AscendC::LoadData(
    dstTensor.data()[dstOffset], srcTensor.data(), scaleTensor.data(), loadDataParams, loadDataMxParams
);
```

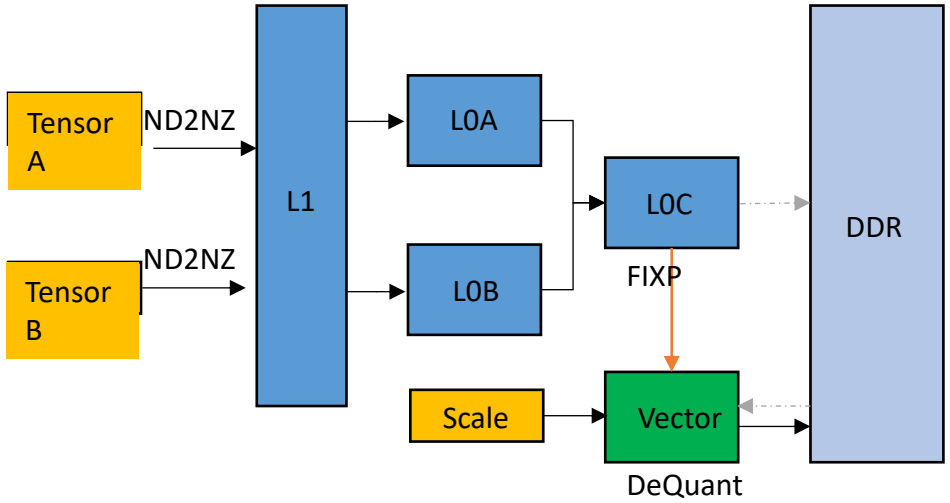
设置LoadData2DMx指令搬运参数

调用LoadData2DMx指令

CANN

Ascend 950新特性支持：C-V新通路

➢ **以L0C到UB通路为例**



GMMDeQuant样例计算过程示意图

上一代芯片数据通路

下一代芯片新增数据通路

Fixpipe指令资料描述片段：

•通路CO1->UB/C1，不使能tensor量化功能：

```
template <typename T, typename U, const FixpipeConfig& config = CFG_ROW_MAJOR>
__aicore__ inline void Fixpipe(const LocalTensor<T>& dst, const LocalTensor<U>&
src, const FixpipeParamsC310<config.format>& intriParams)
```

Kernel层 AIC代码片段：

构造Vector上的LocalTensor用于存放输出矩阵C

```
auto ubGmmTensor = tla::MakeTensor(ubGmmResList[ubListId], layoutGmm, Arch::PositionUB{});

AscendC::CrossCoreWaitFlag<AIC_SYNC_AIV_MODE_4, PIPE_FIX>(AIV_SYNC_AIC_FLAG + ubListId);
AscendC::CrossCoreWaitFlag<AIC_SYNC_AIV_MODE_4, PIPE_FIX>(AIV_SYNC_AIC_FLAG + FLAG_ID_MAX + ubListId);
blockMmadTla(
    gmATile, gmBTile, ubGmmTensor,
    blockShape
);
AscendC::CrossCoreSetFlag<AIC_SYNC_AIV_MODE_4, PIPE_FIX>(AIC_SYNC_AIV_FLAG + ubListId);
AscendC::CrossCoreSetFlag<AIC_SYNC_AIV_MODE_4, PIPE_FIX>(AIC_SYNC_AIV_FLAG + FLAG_ID_MAX + ubListId);
```

传入至BlockMmad接口，将结果矩阵C从L0C输出至UB

Kernel层 AIV代码片段：

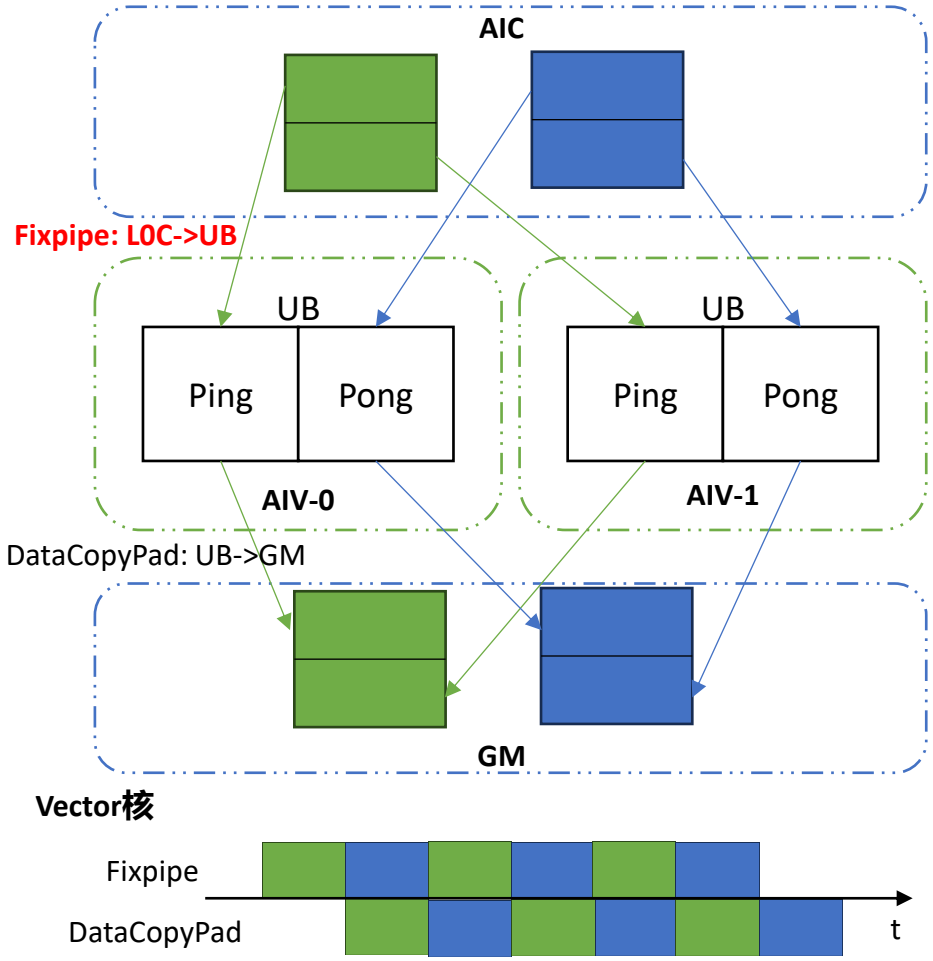
```
AscendC::CrossCoreWaitFlag<AIC_SYNC_AIV_MODE_4, PIPE_V>(AIC_SYNC_AIV_FLAG + ubListId);
blockEpilogue(
    gmDequantTile,
    ubGmmTensor,
    gmScaleTile,
    gmPerTokenTile,
    ubListId
);
AscendC::CrossCoreSetFlag<AIC_SYNC_AIV_MODE_4, PIPE_MTE3>(AIV_SYNC_AIC_FLAG + ubListId);
```

传入至BlockEpilogue接口，执行DeQuant操作

CANN

Ascend 950新特性支持：指令新特性增强

➢ **以Fixpipe 支持DualDst为例**



Fixpipe_opt样例流水示意图

Tile层 代码片段：

```
// Fixpipe auxiliary arguments
intriParams.quantPre = quantPre;
intriParams.reluEn = reluEn;
intriParams.unitFlag = unitFlag;
intriParams.dualDstCtl = 1;

auto dstOffset = dstTensor.layout()(dstTensor.coord());
auto srcOffset = srcTensor.layout()(srcTensor.coord());

// Call AscendC Fixpipe
AscendC::Fixpipe<ElementDst, ElementSrc, CFG_ROW_MAJOR_UB>(
    dstTensor.data()[dstOffset], srcTensor.data()[srcOffset], intriParams);
```

设置Fixpipe指令搬运参数

调用Fixpipe指令

Ascend 950新特性支持：指令新特性增强

➢ **以Fixpipe 支持DualDst为例**

BlockEpilogue 代码片段：

```
template <class TensorOut, class TensorIn>
CATLASS_DEVICE
void operator()(TensorOut& tensorOut, TensorIn& tensorIn, bool splitM)
{
    int64_t blockShapeM = tla::get<0>(tensorOut.shape());
    int64_t blockShapeN = tla::get<1>(tensorOut.shape());
    int64_t halfBlockShapeM = CeilDiv(blockShapeM, AscendC::GetTaskRation());
    if (splitM) {
        blockShapeM = (static_cast<uint64_t>(blockShapeM) & 1UL) > 0UL
                      ? (halfBlockShapeM - AscendC::GetSubBlockIdx())
                      : halfBlockShapeM;
    }

    // real copy data size
    int64_t inputSize = blockShapeM * blockShapeN;
    if (inputSize <= 0) {
        return;
    }

    auto tensorTileOut = GetTile(
        tensorOut,
        tla::MakeCoord(halfBlockShapeM * (AscendC::GetSubBlockIdx() & 0x1), 0),
        tla::MakeShape(blockShapeM, blockShapeN)
    );

    using CopyUbToGm = Epilogue::Tile::CopyUb2GmTla<ArchTag, TensorIn, decltype(tensorTile
    CopyUbToGm copyUbToGm;
    copyUbToGm(tensorTileOut, tensorIn);
}
```

获取当前AIV的输出Tensor切块

Kernel层 代码片段：

```
if ASCEND_IS_AIC {
    AscendC::CrossCoreWaitFlag<AIC_SYNC_AIV_MODE_4, PIPE_FIX>(AIV_SYNC_AIC_FLAG + (pingPongIdx));
    if (enableDualDst) {
        AscendC::CrossCoreWaitFlag<AIC_SYNC_AIV_MODE_4, PIPE_FIX>(
            AIV_SYNC_AIC_FLAG + (pingPongIdx) + FLAG_ID_MAX);
    }

    blockMmadOp(
        cLocalTensor,
        aTileTensor,
        bTileTensor,
        blockShape,
        tileIdx == curBlockIdx);
    AscendC::CrossCoreSetFlag<AIC_SYNC_AIV_MODE_4, PIPE_FIX>(AIC_SYNC_AIV_FLAG + (pingPongIdx));
    if (enableDualDst) {
        AscendC::CrossCoreSetFlag<AIC_SYNC_AIV_MODE_4, PIPE_FIX>(
            AIC_SYNC_AIV_FLAG + (pingPongIdx) + FLAG_ID_MAX);
    }
}
if ASCEND_IS_AIV {
    // Synchronize with aic
    AscendC::CrossCoreWaitFlag<AIC_SYNC_AIV_MODE_4, PIPE_MTE3>(AIC_SYNC_AIV_FLAG + (pingPongIdx));
    // Calulate epilogue
    epilogueOp(cTileTensor, cLocalTensor, enableDualDst);
    // Notify aic
    AscendC::CrossCoreSetFlag<AIC_SYNC_AIV_MODE_4, PIPE_MTE3>(AIV_SYNC_AIC_FLAG + (pingPongIdx));
}
```

设置AIC核和AIV-1核之间的核间同步

CANN

CATLASS新增特性：TLA(Tensor Layout Abstraction)新增接口支持便捷Tensor访问

功能与定位

TLA(Tensor Layout Abstraction)作为CATLASS的一种数据结构，提供了对基础Tensor的封装，

新版扩展接口

- MakeTensor
- MakeTensorLike
- GetTile
- TileView

优势

- 支持基块切分逻辑，隐藏尾块处理，提升编程易用性
- 提供编译优化能力，降低运行时开销

GetTile

- **功能：**GetTile 用于从父 tensor 上切出一个 tile **视图**

- **接口定义：**

```
template <class Tensor, class Coord, class Shape>
auto GetTile(
        Tensor const& tensor,
        Coord const& coord, // 元素坐标（不是 tile 坐标）
       Shape const& shape); // tileShape：用于内存布局计算的“期望尺寸”
```

- **使用示例：**

```
// Locate the current tile on L0A
auto l0ATile = l0ATensorList[l0AListId];
auto layoutAInL0 = tla::MakeLayout<ElementA, LayoutTagL0A>(mL0Actual,
kL0Actual);
auto tensorL0A = tla::MakeTensor(l0ATile, layoutAInL0, Arch::PositionL0A{});
// Locate the current tile of matrix A on L1
auto tensorTileL1A = GetTile(
    tensorL1A,
    mL0Idx * L0_TILE_M, kL0Idx * L0_TILE_K,
    mL0Actual, kL0Actual);
// … …
// Load current tile from L1 to L0A
copyL1ToL0A(tensorL0A, tensorTileL1A);
// … …
```

TileView

- **功能：**按输入Tile坐标对Tensor进行逻辑切分

- **接口定义：**

```
template <class TensorT, class TileCoord, class TileShape>
auto TileView(
                TensorT const& tensor,
                TileCoord const& tileCoord, // tile 单位坐标
                TileShape const& tileShape); // tile 尺寸
```

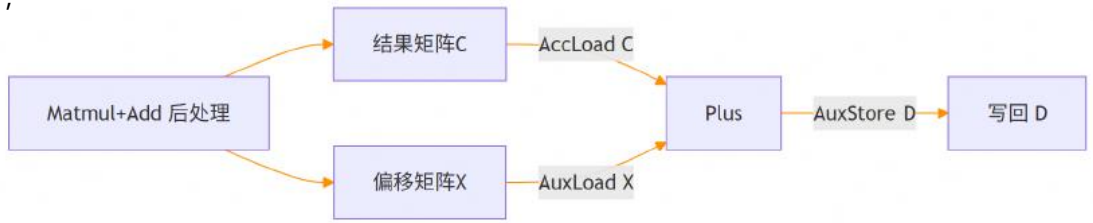
- **使用示例：**

```
// Locate the current tile on L0A
auto l0ATile = l0ATensorList[l0AListId];
// Locate the current tile of matrix A on L1
// Take a (L0_TILE_M, L0_TILE_K) tile from the current L1A tile (tile coordinates
within L1 tile).
auto tensorTileL1A = tla::TileView(
  tensorL1A,
  tla::MakeCoord(mPartIdx, kPartIdx),
  tla::MakeShape(Int<L0_TILE_M>{}, Int<L0_TILE_K>{})
);
auto tensorL0A = tla::MakeTensorLike<LayoutTagL0A>(l0ATile, tensorTileL1A,
Arch::PositionL0A{});
// … …
// Load current tile from L1 to L0A
copyL1ToL0A(tensorL0A, tensorTileL1A);
// … …
```

CATLASS新增特性： EVG(Epilogue Visitor Graph)支持快速Epilogue组合

- 功能：EVG是CATLASS中用于GEMM后处理(Epilogue)编写的声明式框架,后处理操作中的常见环节（如加法计算、类型转换、广播等）被抽象为为可组合的模板节点，并进一步以树形或拓扑结构拼接形成计算图
- 支持数据结构：TreeVisitor和TopilogicalVisitor
  - TreeVisitor：支持属性结构的节点组合，编写和维护更简单，适合没有共享子表达式的场景
  - TopologicalVisitor：支持DAG拓扑结构，当存在表达式一对多的情况需要用DAG进行表达，适合有共享子表达式的场景

```
using EVG = Epilogue::Fusion::TreeVisitor<
      Epilogue::Fusion::VisitorAuxStore<ElementC, LayoutC>,
      Epilogue::Fusion::TreeVisitor<
         Epilogue::Fusion::VisitorCompute<Epilogue::Fusion::Add, ElementC>,
         Epilogue::Fusion::VisitorAccLoad<ElementC>, // 加载 C (workspace)
         Epilogue::Fusion::VisitorAuxLoad<ElementC, LayoutC>   // 加载 X
      >
   >;

   // Block level, define BlockEpilogue with EVG
   using BlockEpilogue = Epilogue::Block::BlockEpilogue<
      Epilogue::EpilogueVisitor<>,
      ArchTag,
      Int<computeLength>,
      EVG,
      ElementC
   >;
```




```
// 节点顺序:
   // 0-AccLoad, 1-Compute1(2X), 2-Compute2(Exp(2X)),
   // 3-Compute3(Exp(2X) + 1), 4-Compute4(Exp(2X) - 1), 5-Compute5(Compute3 /
Compute4), 6-Store
   using Edges = tla::tuple<
      tla::seq<>,        // 0: AccLoad 无子节点
      tla::seq<0>,       // 1: 依赖 AccLoad-->2X
      tla::seq<1>,       // 2: 依赖 Compute1-->Exp(2X)
      tla::seq<2>,       // 3: 依赖 Compute2-->(Exp(2X) - 1)
      tla::seq<2>,       // 4: 依赖 Compute2-->(Exp(2X) + 1)
      tla::seq<3, 4>,    // 5: 依赖 Compute3 与 Compute4-->(Compute3 / Compute4)
      tla::seq<5>        // 6: Store 依赖 Compute5
   >;

   using EVG = Epilogue::Fusion::TopologicalVisitor<
      Edges,
      Epilogue::Fusion::VisitorAccLoad<ElementC>,
      Epilogue::Fusion::VisitorCompute<Epilogue::Fusion::Muls, ElementC, ElementC>,
      Epilogue::Fusion::VisitorCompute<Epilogue::Fusion::Exp, ElementC>,
      Epilogue::Fusion::VisitorCompute<Epilogue::Fusion::Adds, ElementC, ElementC>,
      Epilogue::Fusion::VisitorCompute<Epilogue::Fusion::Adds, ElementC, ElementC>,
      Epilogue::Fusion::VisitorCompute<Epilogue::Fusion::Div, ElementC>,
      Epilogue::Fusion::VisitorAuxStore<ElementC, LayoutC>
   >;
```

目录

Part 1 算子开发旅程

Part 2 Cube算子模板库 CATLASS

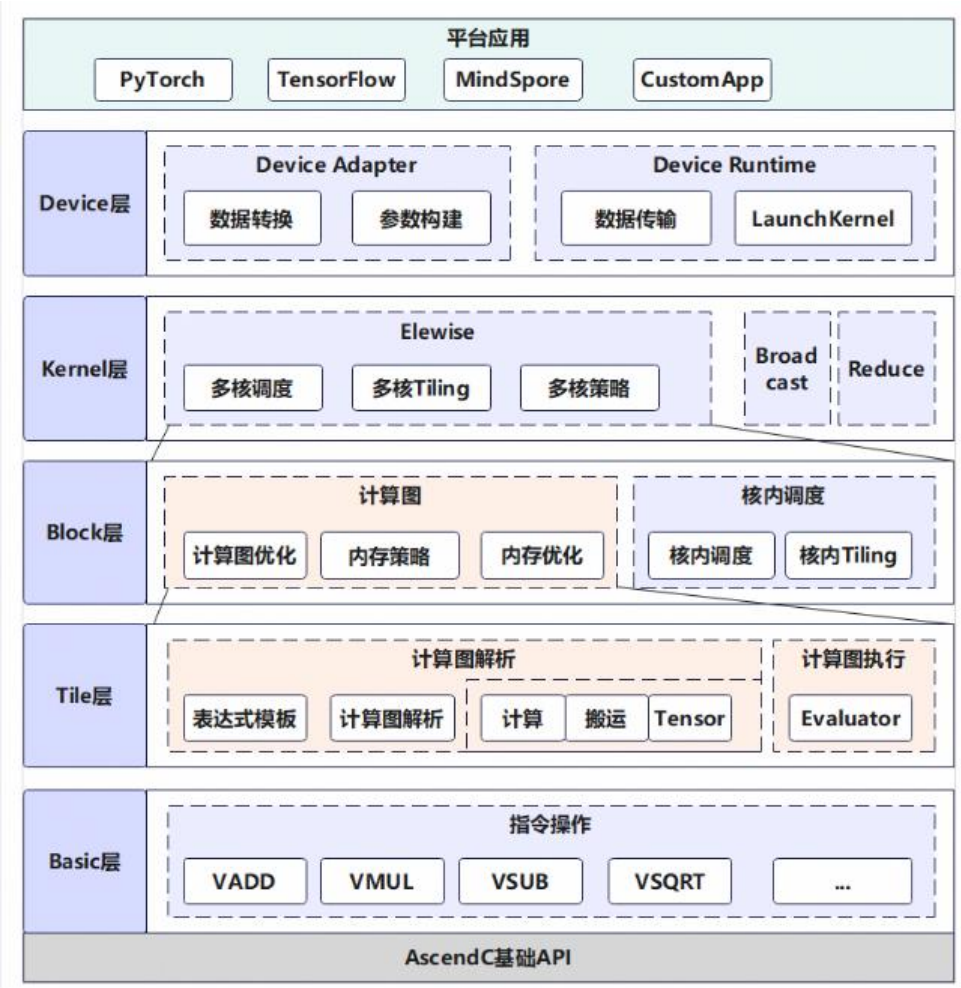
Part 3 Vector算子模板库 ATVOSS

Part 4 <<<>>>直调

Part 5 仿真工具

ATVOSS是Vector算子模板库，致力于为昇腾硬件上的Vector类融合算子提供极简、高效、高性能、高拓展的编程方式

ATVOSS：Ascend C Templates for Vector Operator Subroutines



Device支持<<<>>>调用，可**省去aclnn接口和GEIR**

按NPU硬件编程层级细分成kernel层、block层、tile层，与匹配，每层支持用户定制与调优

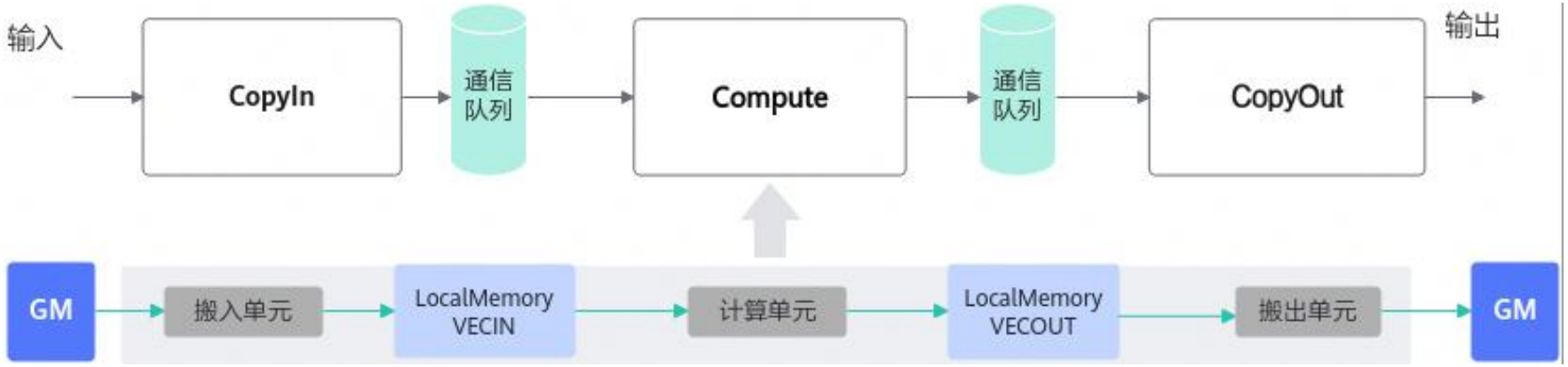
tile层compute描述更简洁，可视感更强，开发者只需要关注算子的数学逻辑表达，无效关心硬件实现的细节

https://gitcode.com/cann

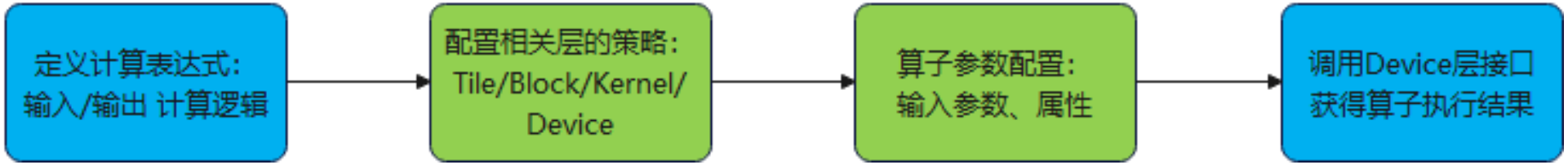
CANN

ATVOSS编程与手写算子的区别

- 手写算子



- ATVOSS写算子



ATVOSS算子示例

```
template <typename T>
struct AbsConfig {
    using Dtype = T;
    using TileShape = Atvoss::Shape<TILE_SIZE>;
    struct AbsCompute {
        template <template <typename> class Tensor>
        __host_aicore__ constexpr auto Compute() const
        {
            auto in1 = Atvoss::PlaceHolder<1, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
            auto out1 = Atvoss::PlaceHolder<2, Tensor<Dtype>, Atvoss::ParamUsage::OUT>();
            return (out1 = Abs(in1));
        };
    };

    static constexpr Atvoss::Ele::DefaultBlockPolicy<TileShape> blockPolicy{
        TileShape{}
    };

    using BlockOp = Atvoss::Ele::BlockBuilder<
        AbsCompute,
        Atvoss::Arch::DAV_3510,
        blockPolicy,
        Atvoss::Ele::DefaultBlockConfig>;

    using KernelOp = Atvoss::Ele::KernelBuilder<
        BlockOp,
         Atvoss::Ele::defaultKernelPolicy>;

    using DeviceOp = Atvoss::DeviceAdapter<KernelOp>;
};
```

```
template <typename T>
struct MulAddConfig {
    using Dtype = T;
    using TileShape = Atvoss::Shape<TILE_SIZE>;
    struct AbsCompute {
        template <template <typename> class Tensor>
        __host_aicore__ constexpr auto Compute() const
        {
            auto in1 = PlaceHodler<1, Tensor<T>, Atvoss::Arguments::IN>();
            auto in2 = PlaceHodler<2, Tensor<T>, Atvoss::Arguments::IN>();
            auto in3 = PlaceHodler<3, Tensor<T>, Atvoss::Arguments::IN>();
            auto out = PlaceHodler<4, Tensor<T>, Atvoss::Arguments::OUT>();
            return out = in1 * in2 + in3;
        };
    };

    static constexpr Atvoss::Ele::DefaultBlockPolicy<TileShape> blockPolicy{
        TileShape{}
    };

    using BlockOp = Atvoss::Ele::BlockBuilder<
        AbsCompute,
        Atvoss::Arch::DAV_3510,
        blockPolicy,
        Atvoss::Ele::DefaultBlockConfig>;

    using KernelOp = Atvoss::Ele::KernelBuilder<
        BlockOp,
         Atvoss::Ele::defaultKernelPolicy>;

    using DeviceOp = Atvoss::DeviceAdapter<KernelOp>;
};
```

ATVOSS易用性改进-数学表达式简化

```
/* 公式:  out = {Abs[(in1 + in2) * in3 + in3] + in4} * in2 */
struct MyCompute {
    template <template <typename> class Tensor>
    __host_aicore__ constexpr auto Compute() const
    {
        auto in1 = Atvoss::PlaceHolder<1, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in2 = Atvoss::PlaceHolder<2, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in3 = Atvoss::PlaceHolder<3, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in4 = Atvoss::PlaceHolder<4, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto out1 = Atvoss::PlaceHolder<5, Tensor<Dtype>, Atvoss::ParamUsage::OUT>();
        auto temp1 = Atvoss::PlaceHolderTmpLike<1, Tensor<DtypeOther>>(in1);
        auto temp2 = Atvoss::PlaceHolderTmpLike<1, Tensor<DtypeOther>>(in1);

        return (temp1 = in1 + in2,
                temp2 = temp1 * in3,
                temp1 = temp2 + in3,
                temp2 = Abs(temp1),
                out = temp2 * in2)
    };
};
```

```
/* 公式:  out = {Abs[(in1 + in2) * in3 + in3] + in4} * in2 */
struct MyCompute {
    template <template <typename> class Tensor>
    __host_aicore__ constexpr auto Compute() const
    {
        auto in1 = Atvoss::PlaceHolder<1, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in2 = Atvoss::PlaceHolder<2, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in3 = Atvoss::PlaceHolder<3, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in4 = Atvoss::PlaceHolder<4, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto out1 = Atvoss::PlaceHolder<5, Tensor<Dtype>, Atvoss::ParamUsage::OUT>();
        auto _1 = (in1 + in2) * in3 + in3;
        auto _2 = Abs(_1) + in4;
        return out1 = _2 * in2;
    };
};
```

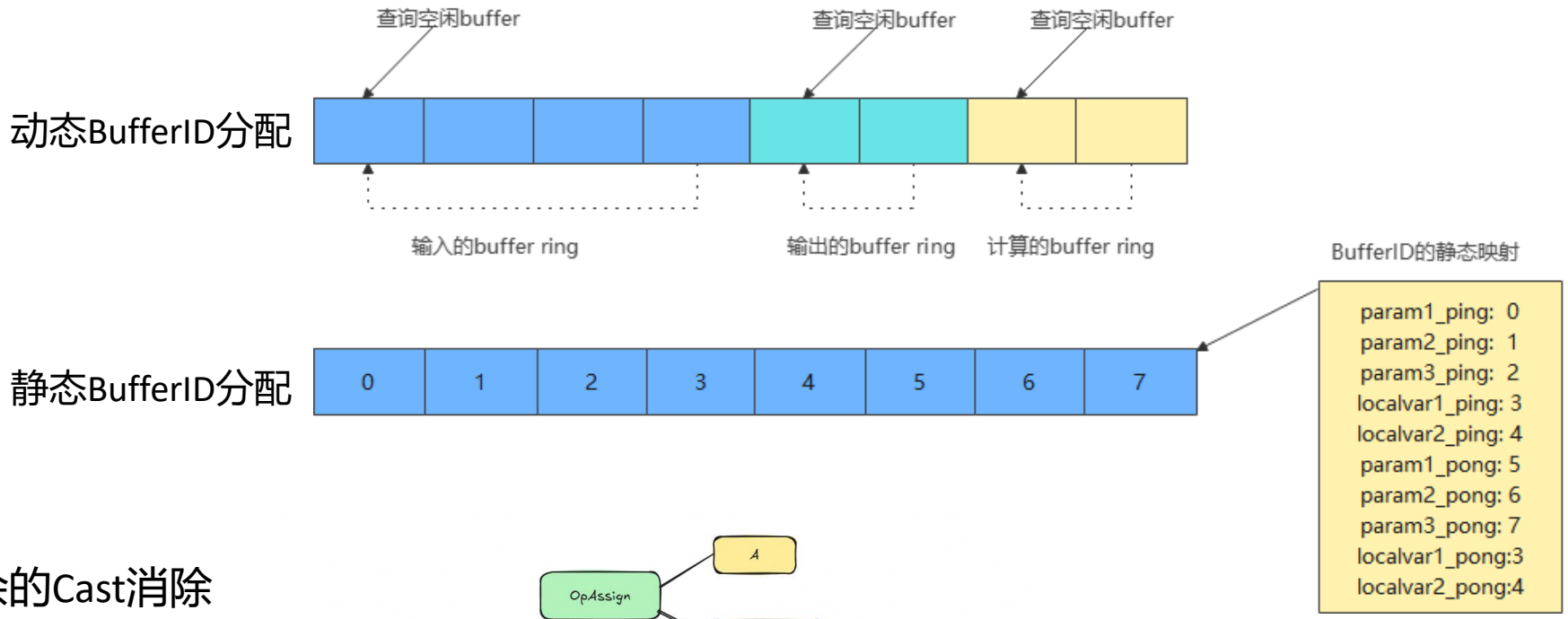
```
/* 公式:  out = {Abs[(in1 + in2) * in3 + in3] + in4} * in2 */
struct MyCompute {
    template <template <typename> class Tensor>
    __host_aicore__ constexpr auto Compute() const
    {
        auto in1 = Atvoss::PlaceHolder<1, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in2 = Atvoss::PlaceHolder<2, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in3 = Atvoss::PlaceHolder<3, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto in4 = Atvoss::PlaceHolder<4, Tensor<Dtype>, Atvoss::ParamUsage::IN>();
        auto out1 = Atvoss::PlaceHolder<5, Tensor<Dtype>, Atvoss::ParamUsage::OUT>();

        return out1 = (Abs((in1 + in2) * in3 + in3) + in4) * in2;
    };
};
```

数学表达式简化：只需关注数学逻辑的表达，支持长表达式

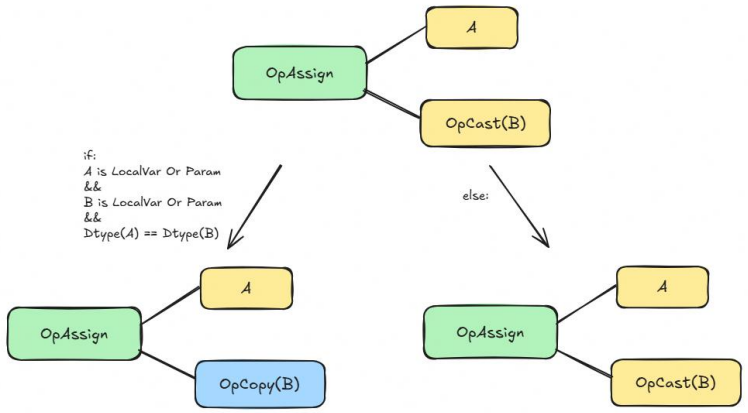
CANN

ATVOSS性能优化

- 静态BufferID分配：根据算子的计算图提前规划运算需要的BufferID，减少scalar操作导致的流水打断



- 冗余的Cast消除



CANN

ATVOSS算子的调用

```
template <typename T>
void Run()
{
   // --- 资源初始化: ACL 初始化、设置 device、创建 Context、 创建 Stream---
  ......
   CHECK_ACL_RET(aclInit(nullptr));
   CHECK_ACL_RET(aclrtSetDevice(deviceId));
   CHECK_ACL_RET(aclrtCreateContext(&context, deviceId));
   CHECK_ACL_RET(aclrtCreateStream(&stream));

   // --- 准备数据、分配设备内存、Device 数据拷贝 ---
   ......
   CHECK_ACL_RET(aclrtMalloc(&rawInput, inputSize, ACL_MEM_MALLOC_HUGE_FIRST));
   CHECK_ACL_RET(aclrtMalloc(&rawOutput, outputSize,ACL_MEM_MALLOC_HUGE_FIRST));
   CHECK_ACL_RET(aclrtMemcpy(deviceInput, inputSize, hostInput.data(), inputSize,
ACL_MEMCPY_HOST_TO_DEVICE));

   // --- 构造 Tensor 并执行算子 ---
   uint32_t shapeArray[MAX_DIM] = {0};
   std::copy(shape.begin(), shape.end(), shapeArray);
   Atvoss::Tensor<T> t1(deviceInput, shapeArray, shape.size());
   Atvoss::Tensor<T> t2(deviceOutput, shapeArray, shape.size());
   auto arguments = Atvoss::ArgumentsBuilder{}.inputOutput(t1, t2).build();
   using DeviceOp = typename AbsConfig<T>::DeviceOp;
   DeviceOp deviceOp;
   deviceOp.Run(arguments, stream);

   // --- 同步拷回结果并验证 ---
   CHECK_ACL_RET(aclrtSynchronizeStream(stream));
   std::vector<T> hostOutput(shapeSize);
   CHECK_ACL_RET(aclrtMemcpy(hostOutput.data(), outputSize, deviceOutput, outputSize,
ACL_MEMCPY_DEVICE_TO_HOST));
    ......
}
```

目录

Part 1 算子开发旅程

Part 2 Cube算子模板库 CATLASS

Part 3 Vector算子模板库 ATVOSS

Part 4 <<<>>>直调

Part 5 仿真工具

ASC支持异构编译+<<<>>>直调，提供与业界一致编程体验

➢ **Host/Device混合编译，不需要手动区分Host/Device代码！一键式完成异构编译，核函数支持复杂模板！**

```
#include "kernel_operator.h"
#include "acl/acl.h"
template<class T>
__global__ __vector__ void hello_world(T value) {

    Ascend C::printf("Hello World:  %d!!!\n",  value);
}

int32_t main(int argc, char const *argv[])
{
  aclInit(nullptr);
  int32_t deviceId = 0;
  aclrtSetDevice(deviceId);
  aclrtStream stream = nullptr;
  aclrtCreateStream(&stream);
  constexpr uint32_t blockNum = 8;
  hello_world<int><<< blockNum, nullptr, stream>>>(10);
  aclrtSynchronizeStream(stream);
  aclrtDestroyStream(stream);
  aclrtResetDevice(deviceId);
  aclFinalize();
  return 0;
}
```

设备侧编译

Device代码 → fatbin.o

主机侧编译

Host代码 → host.o

链接 → bin/.so/.a

统一编译入口，命令行编译使用示例：

bisheng -xasc hello_world.cpp -o hello --npu-arch=dav-3510

竞品：nvcc -arch=sm_61 -o hello_world hello_world.cu

算子工程 降维为 编程扩展能力

```
cmake_minimum_required(VERSION 3.16)
project(cmake_and_cuda LANGUAGES CXX CUDA)
```

↕

```
cmake_minimum_required(VERSION 3.16)
find_package(ASC REQUIRED)
project(kernel_samples LANGUAGES CXX ASC)
```

➢ **工程极大简化！**

```
├── cmake // CMake配置文件（必需）
│   ├── cpu_lib.cmake
│   └── npu_lib.cmake
├── CMakeLists.txt // CMakeLists.txt（必需）
├── data_utils.h
├── main.cpp // Host侧调用程序 （必需）
├── matmul_leakyrelu_custom.cpp // mix算子计算逻辑（必需）
├── matmul_leakyrelu_custom_tiling.cpp // Host侧tiling计算逻辑（必需）
├── README.md
├── run.sh // 一键运行脚本（可选）
└── scripts // 验证脚本
    ├── gen_data.py
    └── verify_result.py
```

→

```
├── CMakeLists.txt // CMakeLists.txt（可选，可直接命令行编译）
├── data_utils.h
├── matmul_leakyrelu.asc // mix算子Host侧调用代码，Kernel实现
├── README.md
├── run.sh // 一键运行脚本（可选）
└── scripts // 验证脚本
    ├── gen_data.py
    └── verify_result.py
```

核心交付件完全对标竞品

```
/cuda-samples-master/Samples/0_Introduction/matrixMul# ll

062 Sep  5 16:29 CMakeLists.txt
184 Sep  5 16:29 matrixMul.cu
052 Sep  5 16:29 README.md
```

注：本页案例来源：

https://gitcode.com/cann/asc-devkit/tree/master/examples/00_introduction/00_helloworld

https://gitcode.com/cann/asc-devkit/tree/master/examples/00_introduction/03_matmulleakyrelu

目录

Part 1 算子开发旅程

Part 2 Cube算子模板库 CATLASS

Part 3 Vector算子模板库 ATVOSS

Part 4 <<<>>>直调

Part 5 仿真工具

利用仿真工具进行验证

- 编译生成算子测试的可执行文件

编译命令：bash scripts/build.sh -soc=ascend950 --example abs
生成的文件：output/bin/abs

- 利用仿真工具进行验证

设置环境变量：source ~/Ascend/cann/bin/setenv.bash
执行命令：**cannsim** record **./abs** -u **"--shape=32,32 --type_input=f32"** -s Ascend950 --gen-report
执行结果查看：view cannsim_20260305150418_abs/cannsim.log

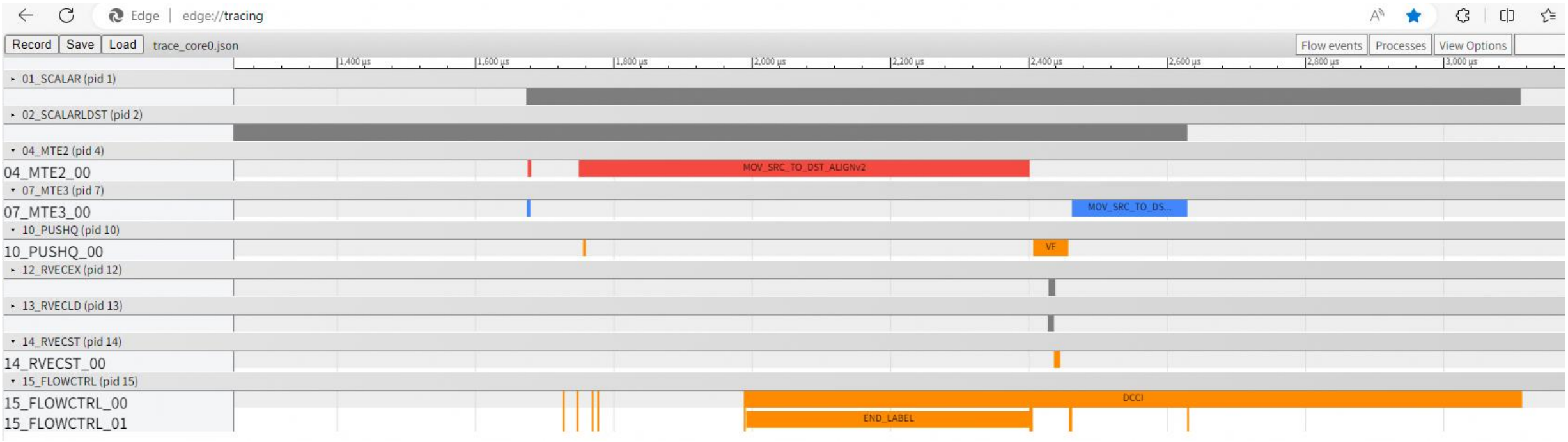
tips：如果cannsim的版本还不支持-u参数，那就把./abs --shape=32,32 --type_input=f32 放到一个shell脚本中(abs.sh)，执行cannsim record ./abs.sh -s Ascend950 --gen-report

```
SoC sub 0 3 all tasks are finished!
[Hardware] parallel simulation finish. sim time: 1.113503s, cycle: 419, speed: 0.376290KHz
SoC sub 0 2 all tasks are finished!
[Hardware] parallel simulation finish. sim time: 1.435089s, cycle: 420, speed: 0.292665KHz
Accuracy verification passed.
Parallel Simulation: All threads are joined.
ESL_TOP: parallel threads joined
ESL_TOP: system destructed
[INFO] Model stopped successfully.
```


CANN

利用仿真工具进行验证

流水图文件：xxxx/cannsim_xxx_abs/report/**trace_core0.json**
在edge浏览器中输入"edge://tracing/"或是在Chrome浏览器中输入"chrome://tracing", 把**trace_core0.json**文件拖进去



CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN